SmartCollect Phoenix - Reinforcement Learning

Pipeline de aprendizado por reforco que melhora o agente continuamente com base em outcomes de negociacao. Calibrado para 5k conversas/dia (~150k/mes) com o Santander, usando rewards verificaveis e progressao em 3 fases: Training-Free RL, GRPO com rLLM, e AgentFly avancado.

1. Problema

1.1 Dados Desperdicados

O SmartCollect ja rastreia outcomes via Kafka (ver `13-OBSERVABILITY.md`):

Evento	Kafka Topic	Status
`agreement_created`	`smartcollect.telemetry`	Emitido
`agreement_failed`	`smartcollect.telemetry`	Emitido
`offers_presented`	`smartcollect.telemetry`	Emitido
`conversation_ended`	`smartcollect.telemetry`	Emitido
`offer_rejected`	`smartcollect.telemetry`	Emitido

Porem, nenhum desses dados e usado para melhorar o comportamento do agente. A ~5k conversas/dia, sao **~150k conversas/mes de dados desperdicados**.

1.2 Estrategias Fixas

A selecao de estrategia e baseada em regras fixas (ver `16-COGNITIVE-ARCHITECTURE.md` secao 4.2, `_rule_based_fallback`). Nao ha adaptacao baseada em: o que funcionou com clientes similares, qual carteira tem melhor resposta a qual abordagem, ou quais estrategias tem maior win rate.

1.3 Oportunidade

Metrica	Valor	Significado para RL
Conversas/dia	~5.000	Dados de treino abundantes
Turnos/conversa	~15-20	Trajetorias multi-turn ricas
Taxa de acordo (estimada)	~30-40%	Espaco para melhoria
Portfolios distintos	4-5	Policies especializadas por segmento
Reward verificavel	Sim	Sem necessidade de reward model

2. Reward Signal Design

2.1 ConversationReward

```python
from dataclasses import dataclass
from decimal import Decimal


@dataclass(frozen=True)
class ConversationReward:
    agreement_created: float
    agreement_paid: float
    customer_satisfaction: float
    efficiency_bonus: float
    escalation_penalty: float
    breach_penalty: float
    abandonment_penalty: float

    @property
    def total(self) -> Decimal:
        return Decimal(str(
            self.agreement_created
            + self.agreement_paid
            + self.customer_satisfaction
            + self.efficiency_bonus
            + self.escalation_penalty
            + self.breach_penalty
            + self.abandonment_penalty
        ))
```

2.2 Componentes do Reward

Componente	Valor	Quando	Fonte
`agreement_created`	+1.0	Acordo gerado no RaaS	`emit_agreement_created`
`agreement_paid`	+2.0	Acordo pago (boleto quitado)	Callback de pagamento (dias depois)
`customer_satisfaction`	+0.5	NPS/CSAT positivo pos-atendimento	Pesquisa pos-conversa

Componente	Valor	Quando	Fonte
`efficiency_bonus`	+0.2	Conversa < mediana de turnos	`conversation_ended` com `turn_count`
`escalation_penalty`	-0.1	Escalou para humano	`human_fallback` node
`breach_penalty`	-0.5	Acordo quebrado (inadimplencia)	Evento de breach (semanas depois)
`abandonment_penalty`	-0.3	Cliente abandonou apos negociacao iniciada	Inactivity timeout

2.3 Reward Temporal

Nem todos os componentes estao disponiveis no momento do encerramento:

```
T=0 (conversa)     T+0..3h (imediato)     T+3..30d (delayed)
──────────────────────────────────────────────────────────────
agreement_created   efficiency_bonus        agreement_paid
escalation_penalty  abandonment_penalty     breach_penalty
                                            customer_satisfaction
```

O `RewardCalculator` computa um **reward parcial** imediato e um **reward final** quando todos os sinais chegam. O Experience Store armazena ambos.

2.4 Rewards Verificaveis

Todos os componentes do reward sao **verificaveis** — nao dependem de julgamento subjetivo:

Componente	Verificacao
Acordo criado	MCP retornou success
Acordo pago	Callback de pagamento
Eficiencia	`turn_count < median_turns`
Escalacao	State `target_agent == "human_fallback"`
Abandono	Inactivity timeout atingido

Componente	Verificacao
Breach	Evento de inadimplencia

Isso significa que GRPO (Group Relative Policy Optimization) pode ser usado diretamente, sem treinar um reward model separado.

3. Experience Store

3.1 NegotiationExperience

```
@dataclass(frozen=True)
class ConversationOutcome:
    status: str
    agreement_id: str | None
    agreement_amount: Decimal | None
    turn_count: int
    duration_seconds: int
    escalated: bool
    exit_reason: str


@dataclass(frozen=True)
class NegotiationExperience:
    conversation_id: str
    portfolio: str
    timestamp: str
    client_profile: dict
    debt_summary: dict
    strategies_used: list[dict]
    cognitive_traces: list[dict]
    outcome: ConversationOutcome
    reward_partial: ConversationReward
    reward_final: ConversationReward | None
    trajectory_tokens: int
```

3.2 Armazenamento

Dual-store para diferentes necessidades:

Store	Uso	Retencao
Store API `("experiences", portfolio)`	Retrieval Learning (busca rapida)	30 dias (rolling window)
PostgreSQL `experiences` table	Training pipeline (GRPO, analytics)	Indefinido

```python
async def save_experience(
    store: BaseStore,
    db: StateStore,
    experience: NegotiationExperience,
) -> None:
    from dataclasses import asdict
    data = asdict(experience)

    await store.aput(
        ("experiences", experience.portfolio),
        experience.conversation_id,
        data,
    )

    await db.insert_experience(data)
```

3.3 Volume Estimado

Metrica	Valor
Experiencias/dia	~5.000
Experiencias/mes	~150.000
Tamanho medio por experiencia	~5KB (sem trajectory completa)
Store API (30 dias)	~750MB
PostgreSQL (6 meses)	~4.5GB

4. Fase 1 - Training-Free RL (Retrieval Learning)

4.1 Conceito

Sem treinar modelo nenhum. O LLM da API (Azure OpenAI) permanece inalterado. O "aprendizado" acontece via selecao de contexto: experiencias passadas com alto reward sao injetadas no prompt para informar decisoes futuras.

Formalmente, isso e equivalente a **Training-Free GRPO** — otimizacao no espaco de contexto em vez do espaco de parametros.

4.2 Retrieval

```
async def retrieve_similar_experiences(
    store: BaseStore,
    client_profile: dict,
    portfolio: str,
    k: int = 5,
) -> list[dict]:
    all_experiences = await store.alist(("experiences", portfolio))

    scored = []
    for item in all_experiences:
        exp = item.value
        if exp.get("reward_partial") is None:
            continue
        similarity = compute_profile_similarity(client_profile, exp["client_profile"])
        reward = exp["reward_partial"]["total"] if isinstance(exp["reward_partial"], di
        score = similarity * 0.6 + min(reward / 3.0, 1.0) * 0.4
        scored.append((score, exp))

    scored.sort(key=lambda x: x[0], reverse=True)
    return [exp for _, exp in scored[:k]]
```

4.3 Profile Similarity

```python
def compute_profile_similarity(profile_a: dict, profile_b: dict) -> float:
    score = 0.0
    weights = 0.0

    if profile_a.get("negotiation_style") == profile_b.get("negotiation_style"):
        score += 0.3
    weights += 0.3

    emotion_a = profile_a.get("dominant_emotion", "neutral")
    emotion_b = profile_b.get("dominant_emotion", "neutral")
    if emotion_a == emotion_b:
        score += 0.2
    weights += 0.2

    cap_a = profile_a.get("price_sensitivity", 0.5)
    cap_b = profile_b.get("price_sensitivity", 0.5)
    score += 0.2 * (1.0 - abs(cap_a - cap_b))
    weights += 0.2

    trust_a = profile_a.get("trust_level", 0.5)
    trust_b = profile_b.get("trust_level", 0.5)
    score += 0.15 * (1.0 - abs(trust_a - trust_b))
    weights += 0.15

    eng_a = profile_a.get("engagement_level", 0.5)
    eng_b = profile_b.get("engagement_level", 0.5)
    score += 0.15 * (1.0 - abs(eng_a - eng_b))
    weights += 0.15

    return score / weights if weights > 0 else 0.0
```

4.4 Injecao no Prompt

O `DynamicPromptBuilder` (ver `08-CONTEXT-ENGINEERING.md`) ganha uma nova secao `<similar_experiences>`:

```python
def _build_experience_context(self, state: FlowState) -> str:
    experiences = state.get("experience_context")
    if not experiences:
        return ""

    sections = []
    for i, exp in enumerate(experiences[:3]):
        reward = exp.get("reward_partial", {})
        total = reward.get("total", 0) if isinstance(reward, dict) else 0
        strategies = [s.get("strategy", "unknown") for s in exp.get("strategies_used",
        outcome = exp.get("outcome", {}).get("status", "unknown")

        sections.append(
            f'<case rank="{i+1}" reward="{total}" outcome="{outcome}">\n'
            f'Estrategias: {", ".join(strategies)}\n'
            f'Resultado: {outcome}\n'
            f'</case>'
        )

    return f'<similar_experiences>\n{"".join(sections)}\n</similar_experiences>'
```

4.5 Timeline

Dia	Atividade
1-2	Implementar `RewardCalculator` + Kafka consumer de reward
3-4	Implementar `ExperienceStore` (Store API + PostgreSQL)
5-7	Implementar retrieval + similarity + injecao no prompt
8-10	Testes + validacao com dados reais

Apos 3 dias de coleta (~15k conversas), o Retrieval Learning ja tem dados suficientes para operar.

5. Fase 2 - GRPO com rLLM

5.1 Conceito

Fine-tuning de modelo open-source (Qwen3-8B ou Llama 4 Scout) usando GRPO (Group Relative Policy Optimization) via framework rLLM. O modelo treinado substitui o LLM da API em nodes especificos.

5.2 Por que GRPO

Algoritmo	Reward Model	Complexidade	Adequacao
PPO	Necessario	Alta	Nao — rewards sao verificaveis, nao subjetivos
DPO	Nao (pares)	Media	Possivel — mas requer pareamento manual
GRPO	**Nao (verificavel)**	**Baixa**	**Ideal — rewards sao verificaveis por definicao**
REINFORCE++	Nao	Baixa	Possivel — baseline simples
RLOO	Nao	Baixa	Possivel — variante eficiente

GRPO compara rewards dentro de um grupo de completions para o mesmo prompt, sem necessidade de reward model separado.

5.3 Integracao com rLLM

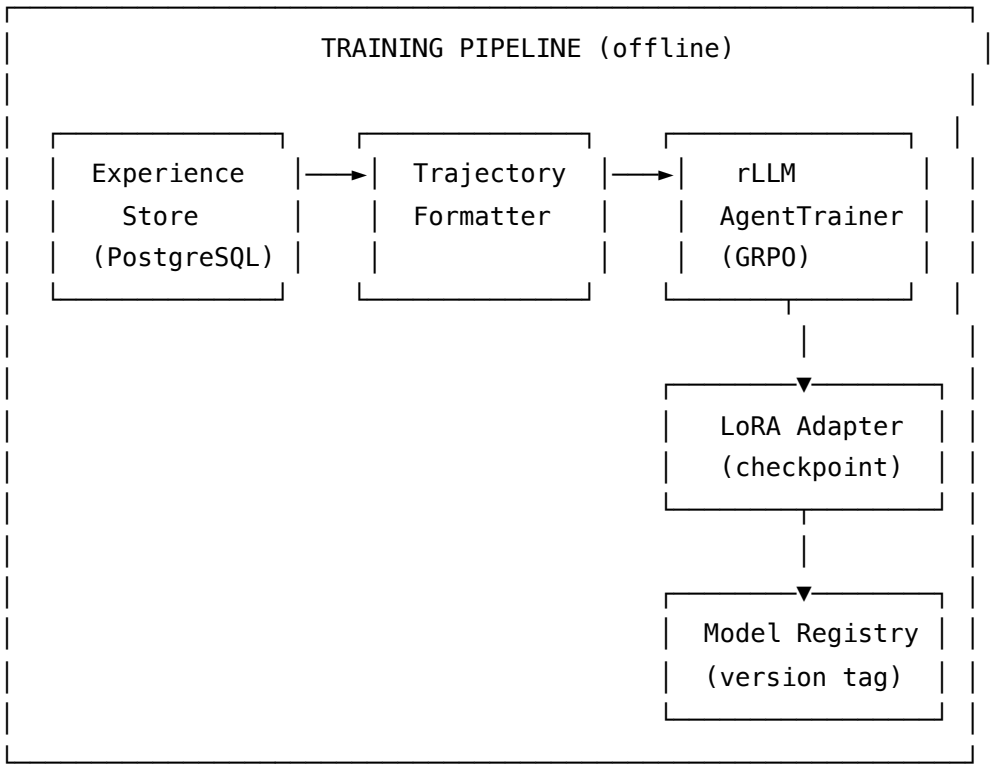
O rLLM integra com LangGraph via **client injection** — zero mudanca no codigo do grafo:

```python
from rllm.sdk import get_chat_client, get_chat_client_async

sync_client = get_chat_client(
    base_url="http://localhost:4000/v1",
    api_key="token",
)
async_client = get_chat_client_async(
    base_url="http://localhost:4000/v1",
    api_key="token",
)

llm = ChatOpenAI(
    model="Qwen/Qwen3-8B",
    client=sync_client,
    async_client=async_client,
)
```

O rLLM captura trajetorias automaticamente via os clients injetados. O `AgentTrainer` orquestra:

1. **Rollout**: Executa o grafo LangGraph N vezes em paralelo
2. **Reward**: Aplica `ConversationReward` em cada trajetoria
3. **Training**: Atualiza pesos do modelo com GRPO
4. **Checkpoint**: Salva LoRA adapter

5.4 Training Pipeline



5.5 Configuracao do Training

```python
from rllm import AgentTrainer, GRPOConfig

config = GRPOConfig(
    model_name="Qwen/Qwen3-8B",
    lora_rank=16,
    lora_alpha=32,
    learning_rate=1e-5,
    num_episodes=1000,
    group_size=4,
    max_steps_per_episode=30,
    kl_coeff=0.05,
    reward_function=compute_conversation_reward,
)

trainer = AgentTrainer(config)
trainer.train(
    trajectory_source=experience_store,
    output_dir="./checkpoints/grpo_v1",
)
```

5.6 Timeline

Semana	Atividade
1	Setup vLLM + rLLM + modelo base (Qwen3-8B)
2	Implementar trajectory formatter (Experience → rLLM format)
3	Primeiro treino GRPO com ~5k trajetorias
4	A/B test modelo treinado vs baseline
5-6	Iteracao + treino com mais dados

6. Fase 3 - AgentFly (Avancado)

6.1 Conceito

O AgentFly resolve um problema que o rLLM nao trata: em conversas multi-turn, quem gerou cada token? O agente gerou a resposta, mas o ambiente injetou a mensagem do cliente, o resultado do MCP, o contexto.

Token-level masking: apenas tokens gerados pelo LLM contribuem para o gradiente. Tokens do ambiente sao mascarados no loss.

6.2 Por que Importa

Numa negociacao de 20 turnos:

Fonte	Tokens	% do Total	Contribui para Gradiente
Respostas do agente	~4.000	~30%	Sim
Mensagens do cliente	~2.000	~15%	Nao (mascarado)
System prompts	~3.000	~22%	Nao (mascarado)
Tool results (MCP)	~2.000	~15%	Nao (mascarado)
Context/history	~2.500	~18%	Nao (mascarado)

Sem masking, o modelo "aprende" sobre tokens que ele nao controla, diluindo o sinal de aprendizado.

6.3 Integracao

```python
from agentfly import AgentEnvironment, tool, reward

@tool
async def get_client_debts(cpf: str) -> dict:
    return await mcp_client.get_client_debts(cpf)

@tool
async def create_arrangement(params: dict) -> dict:
    return await mcp_client.create_arrangement(params)

@reward
def negotiation_reward(trajectory) -> float:
    outcome = trajectory.final_state
    return compute_conversation_reward(outcome).total
```

6.4 Timeline

Trimestre	Atividade
Q2 2026	Avaliacao de necessidade (rLLM suficiente?)
Q3 2026	Implementacao se ganho marginal justificar complexidade

A Fase 3 so e implementada se a Fase 2 (rLLM/GRPO) atingir plateau de performance e houver evidencia de que token-level masking traria ganho significativo.

7. Reward Calculator

7.1 Arquitetura

Kafka consumer dedicado que agrega eventos em rewards por conversa:

```
┌──────────────────┐      ┌──────────────────┐      ┌──────────────┐
│  Kafka Topics    │──────►│ Reward Calculator │──────►│  Experience  │
│                  │      │   (consumer)      │      │    Store     │
│ telemetry        │      │                   │      │              │
│ agreements       │      │  Agrega eventos   │      │ Store API +  │
│ payments         │      │  por conv_id      │      │  PostgreSQL  │
│ surveys          │      │  Computa reward   │      │              │
└──────────────────┘      └──────────────────┘      └──────────────┘
```

7.2 Implementacao

```python
class RewardCalculator:
    def __init__(self, experience_store, median_turns: int = 15):
        self.store = experience_store
        self.median_turns = median_turns
        self._pending: dict[str, dict] = {}

    async def process_event(self, event: dict) -> None:
        conv_id = event["conversation_id"]

        if conv_id not in self._pending:
            self._pending[conv_id] = {
                "events": [],
                "first_seen": datetime.now(),
            }

        self._pending[conv_id]["events"].append(event)
        await self._try_compute(conv_id)

    async def _try_compute(self, conv_id: str) -> None:
        data = self._pending[conv_id]
        events = data["events"]
        event_types = {e["event_type"] for e in events}

        has_terminal = bool(
            event_types & {"agreement_created", "agreement_failed", "conversation_ended
        )

        if not has_terminal:
            return

        reward = self._compute_reward(events)
        await self.store.update_reward(conv_id, reward, partial=True)

    def _compute_reward(self, events: list[dict]) -> ConversationReward:
        agreement = any(e["event_type"] == "agreement_created" for e in events)
        escalated = any(e["event_type"] == "human_fallback" for e in events)
        abandoned = any(
            e["event_type"] == "conversation_ended"
            and e.get("reason") == "inactivity"
            for e in events
        )
```

```
    turn_count = max(
        (e.get("turn_count", 0) for e in events),
        default=0,
    )
    efficient = turn_count < self.median_turns and turn_count > 0

    return ConversationReward(
        agreement_created=1.0 if agreement else 0.0,
        agreement_paid=0.0,
        customer_satisfaction=0.0,
        efficiency_bonus=0.2 if efficient else 0.0,
        escalation_penalty=-0.1 if escalated else 0.0,
        breach_penalty=0.0,
        abandonment_penalty=-0.3 if abandoned else 0.0,
    )

async def process_delayed_event(self, event: dict) -> None:
    conv_id = event["conversation_id"]
    event_type = event["event_type"]

    experience = await self.store.get_experience(conv_id)
    if not experience:
        return

    reward = experience.reward_partial
    updates = {}

    if event_type == "payment_confirmed":
        updates["agreement_paid"] = 2.0

    if event_type == "agreement_breached":
        updates["breach_penalty"] = -0.5

    if event_type == "csat_response":
        score = event.get("score", 3)
        updates["customer_satisfaction"] = 0.5 if score >= 4 else 0.0

    if updates:
        final_reward = ConversationReward(
            agreement_created=reward.agreement_created,
            agreement_paid=updates.get("agreement_paid", reward.agreement_paid),
            customer_satisfaction=updates.get("customer_satisfaction", reward.custo
```

```
        efficiency_bonus=reward.efficiency_bonus,
        escalation_penalty=reward.escalation_penalty,
        breach_penalty=updates.get("breach_penalty", reward.breach_penalty),
        abandonment_penalty=reward.abandonment_penalty,
    )
    await self.store.update_reward(conv_id, final_reward, partial=False)
```

8. Strategy Policy

8.1 Conceito

Pipeline offline que analisa o Experience Store e produz `StrategyConfig` por portfolio/segmento. O output e consumido pelo `decide_strategy()` da Cognitive Architecture (ver `16-COGNITIVE-ARCHITECTURE.md` secao 4.2).

8.2 Pipeline

```
┌──────────────────┐      ┌──────────────────┐      ┌──────────────┐
│  Experience      │──────►│  Policy          │──────►│  Strategy    │
│    Store         │      │  Optimizer       │      │   Config     │
│  (PostgreSQL)    │      │  (offline job)   │      │   (YAML)     │
└──────────────────┘      └──────────────────┘      └──────────────┘
```

8.3 Analise

```
class PolicyOptimizer:
    async def compute_policy(self, portfolio: str) -> StrategyPolicy:
        experiences = await self.store.get_experiences_by_portfolio(
            portfolio, min_date=datetime.now() - timedelta(days=30),
        )

        segments = self._cluster_by_profile(experiences)
        policy = StrategyPolicy(portfolio=portfolio, segments={})

        for segment_id, segment_exps in segments.items():
            strategy_stats = self._compute_strategy_stats(segment_exps)
            policy.segments[segment_id] = SegmentPolicy(
                preferred=self._rank_strategies(strategy_stats),
                avoid=self._identify_failures(strategy_stats),
            )

        return policy

    def _compute_strategy_stats(
        self, experiences: list[NegotiationExperience],
    ) -> dict[str, StrategyStats]:
        stats: dict[str, StrategyStats] = {}

        for exp in experiences:
            for strategy in exp.strategies_used:
                sid = strategy["strategy"]
                if sid not in stats:
                    stats[sid] = StrategyStats(attempts=0, wins=0, total_reward=Decimal
                stats[sid].attempts += 1
                if exp.outcome.status == "agreement_created":
                    stats[sid].wins += 1
                stats[sid].total_reward += exp.reward_partial.total

        return stats
```

8.4 Output: StrategyConfig YAML

```yaml
strategy_policy:
  portfolio: "santander_cartao"
  computed_at: "2026-03-21T10:00:00"
  data_window_days: 30
  total_experiences: 45000

  segments:
    low_income_frustrated:
      preferred_strategies:
        - strategy: "empathy_first"
          weight: 0.45
          win_rate: 0.38
          attempts: 4200
        - strategy: "installment_flexibility"
          weight: 0.35
          win_rate: 0.32
          attempts: 3800
        - strategy: "reframe_value"
          weight: 0.15
          win_rate: 0.25
          attempts: 1200
      avoid_strategies:
        - strategy: "urgency_scarcity"
          win_rate: 0.08
          reason: "perceived as pressure, increases abandonment"
        - strategy: "cash_highlight"
          win_rate: 0.05
          reason: "client cannot pay lump sum"

    cooperative_has_capacity:
      preferred_strategies:
        - strategy: "cash_highlight"
          weight: 0.50
          win_rate: 0.62
          attempts: 5500
        - strategy: "benefit_immediate"
          weight: 0.30
          win_rate: 0.48
          attempts: 3200
      avoid_strategies: []
```

```yaml
resistant_analytical:
  preferred_strategies:
    - strategy: "social_proof"
      weight: 0.35
      win_rate: 0.28
      attempts: 2100
    - strategy: "reframe_value"
      weight: 0.30
      win_rate: 0.25
      attempts: 1800
    - strategy: "empathy_first"
      weight: 0.25
      win_rate: 0.22
      attempts: 1500
  avoid_strategies:
    - strategy: "urgency_scarcity"
      win_rate: 0.10
      reason: "triggers defensive reaction"
```

Este YAML e visivel pelo time de produto e pode ser ajustado manualmente (override de weights, adicao de avoid_strategies).

8.5 Frequencia de Atualizacao

Portfolio	Experiencias/semana	Atualizacao
santander_cartao	~14.000	Semanal
santander_financiamento	~8.400	Semanal
santander_emprestimo	~7.000	Semanal
santander_consignado	~3.500	Quinzenal
escola_mensalidades	~2.100	Quinzenal

9. A/B Testing Framework

9.1 Design

```python
@dataclass(frozen=True)
class ABTest:
    id: str
    portfolio: str
    variants: list[ABVariant]
    start_date: str
    end_date: str | None
    metric: str
    min_conversations_per_arm: int


@dataclass(frozen=True)
class ABVariant:
    name: str
    weight: int
    config: dict


class ABTestManager:
    async def assign_variant(
        self, test_id: str, thread_id: str
    ) -> ABVariant:
        test = self._active_tests[test_id]
        cached = await self.store.aget(("ab_assignments", test_id), thread_id)
        if cached:
            return ABVariant(**cached.value)

        variant = self._weighted_random(test.variants)
        await self.store.aput(("ab_assignments", test_id), thread_id, asdict(variant))
        return variant

    async def get_results(self, test_id: str) -> ABTestResult:
        test = self._active_tests[test_id]
        results = {}

        for variant in test.variants:
            experiences = await self.store.get_experiences_by_variant(
                test.portfolio, test_id, variant.name,
```

```
        )
        results[variant.name] = {
            "conversations": len(experiences),
            "agreements": sum(1 for e in experiences if e.outcome.status == "agreem
            "agreement_rate": self._compute_rate(experiences),
            "avg_reward": self._compute_avg_reward(experiences),
            "avg_turns": self._compute_avg_turns(experiences),
        }

    return ABTestResult(
        test_id=test_id,
        variants=results,
        significant=self._is_significant(results, test.min_conversations_per_arm),
    )
```

9.2 Significancia Estatistica

A 5k conversas/dia com split 50/50:

Conversas por arm	Dias necessarios	Detectable effect size
500	~0.2 dias	8%
1.000	~0.4 dias	5%
2.500	~1 dia	3%
5.000	~2 dias	2%
10.000	~4 dias	1.5%

Com 2.5k por arm (~1 dia), e possivel detectar diferenca de 3% na taxa de acordo com 95% de confianca.

9.3 Auto-Rollback

```python
async def monitor_ab_test(test_id: str) -> None:
    results = await ab_manager.get_results(test_id)

    for variant_name, metrics in results.variants.items():
        if metrics["conversations"] < 500:
            continue

        baseline = results.variants.get("control", {})
        if not baseline:
            continue

        rate_drop = baseline["agreement_rate"] - metrics["agreement_rate"]
        if rate_drop > 0.05:
            logger.info(
                "event=ab_test_auto_stop test_id=%s variant=%s "
                "rate_drop=%.3f threshold=0.05",
                test_id, variant_name, rate_drop,
            )
            await ab_manager.stop_variant(test_id, variant_name)
```

10. Infraestrutura

10.1 Stack de RL

Componente	Tecnologia	Uso
LLM Serving (local)	vLLM	Serve modelo open-source para inference + training rollouts
Training Framework	rLLM	GRPO training com client injection no LangGraph
Model Base	Qwen3-8B ou Llama 4 Scout	Modelo open-source para fine-tune
Adapter	LoRA (rank=16)	Fine-tune leve, deploy rapido

Componente	Tecnologia	Uso
Experiment Tracking	MLflow ou W&B	Track de metricas, hyperparameters, modelos
Model Registry	MLflow Model Registry	Versionamento e deploy de modelos
GPU	1x A100 40GB ou 2x A10G	Training GRPO + vLLM serving

10.2 Deploy de Modelo Treinado

```
┌──────────────┐     ┌──────────────┐     ┌──────────────┐
│ MLflow       │──►│ vLLM Server  │──►│  LangGraph   │
│ Model        │     │ (reload      │     │  (ChatOpenAI │
│ Registry     │     │  adapter)    │     │   client)    │
│ v1.1 → v1.2  │     │              │     │              │
└──────────────┘     └──────────────┘     └──────────────┘
```

O vLLM suporta hot-reload de LoRA adapters sem restart do servidor. Deploy de novo modelo:

1. Training produz LoRA adapter
2. Registra no MLflow com tag de versao
3. vLLM carrega novo adapter
4. A/B test entre versoes via `ABTestManager`

11. Volume e Timelines

11.1 Dados Necessarios vs Tempo

Milestone	Dados Necessarios	Tempo a 5k/dia
Retrieval Learning operacional	500 conversas com reward	~2-3 dias
Primeiro treino GRPO	5.000+ trajetorias	~1 semana
Policy por segmento (1 portfolio)	1.000+ por segmento	~1-2 semanas
DPO (pares de preferencia)	10.000+ pares	~2-3 semanas

Milestone	Dados Necessarios	Tempo a 5k/dia
Policy robusta (todos portfolios)	5.000+ por portfolio	~2-4 semanas
Modelo local em producao	20.000+ trajetorias + A/B validation	~1 mes

11.2 Roadmap

Semana	Fase	Atividade
1-2	Fase 1	RewardCalculator + ExperienceStore + Kafka consumer
3	Fase 1	Retrieval Learning (similarity + prompt injection)
4-5	Fase 1	Validacao com dados reais + metricas
6	Fase 2	Setup vLLM + rLLM + modelo base
7-8	Fase 2	Trajectory formatter + primeiro treino GRPO
9-10	Fase 2	A/B test modelo treinado vs baseline
11-12	Policy	PolicyOptimizer + StrategyConfig por portfolio
13+	Iteracao	Treino continuo semanal, A/B tests, otimizacao

12. Custo

12.1 Cenarios de Custo Mensal

Cenario	LLM API Calls/dia	LLM Local Calls/dia	Custo API/mes	Custo Infra/mes	Total/mes
Baseline (sem cognitive, sem RL)	~150k	0	~$7.5k	$0	~$7.5k
Cognitive seletivo (GPT-4o + mini)	~220k	0	~$8.4k	$0	~$8.4k

Cenario	LLM API Calls/dia	LLM Local Calls/dia	Custo API/mes	Custo Infra/mes	Total/mes
Cognitive + Retrieval (Fase 1)	~220k	0	~$8.4k	$0	~$8.4k
Cognitive + modelo local (Fase 2)	~80k (GPT-4o)	~140k (Qwen3)	~$4k	~$2k (GPU)	~$6k
Modelo local + RL continuo	~50k (GPT-4o)	~170k (Qwen3)	~$2.5k	~$3k (GPU+train)	~$5.5k

12.2 ROI Estimado

Se o RL melhora a taxa de acordo em 5% (de 35% para 40%):

Metrica	Antes	Depois	Delta
Conversas/mes	150.000	150.000	0
Acordos/mes	52.500	60.000	+7.500
Valor medio acordo	R$ 2.000	R$ 2.000	0
Receita recuperada/mes	R$ 105M	R$ 120M	+R$ 15M
Custo adicional/mes	-	-	~R$ 5k

13. Diferenca do Phoenix Base

Aspecto	Phoenix Base	Com Reinforcement Learning
Outcome tracking	Eventos emitidos, nao usados	Rewards computados e armazenados
Estrategia	Regra fixa	Otimizada por dados, per-portfolio
Experiencias passadas	Nao consultadas	Retrieval Learning injeta no prompt

Aspecto	Phoenix Base	Com Reinforcement Learning
Modelo LLM	API only (Azure OpenAI)	API + modelo local fine-tuned
A/B testing	Nao suportado	Framework com auto-rollback
Melhoria continua	Manual (engenharia edita regras)	Automatica (treino semanal)
Policy por segmento	Nao existe	StrategyConfig YAML por portfolio
Custo	~$7.5k/mes	~$5.5k/mes (com modelo local)